

PROGRAMMING IN C – UNIT 3

2 MARKS

1) What is an array? List its types.

Array is a collection of elements with similar data type.

There are 3 types:

- One dimensional array
- Two-dimensional array
- Multi-dimensional array

2) List any four advantages of arrays.

- Arrays represent multiple items of the same data type using a single variable name.
- In arrays, the elements can be accessed randomly by using the index number.
- Arrays allocate memory in contiguous memory locations for all its elements. Hence there is no chance of extra memory being allocated in case of arrays. This avoids memory overflow or shortage of memory in arrays.
- Using arrays, other data structures like linked lists, stacks, queues, trees, graphs etc can be implemented.
- Two-dimensional arrays are used to represent matrices.

3) Differentiate one-dimensional and two-dimensional arrays

- One dimensional array

It is a list of items that has one variable name and one subscript to access the items. It represent the list of elements.

- Two-dimensional array

It is an array that has single variable name and two subscripts for accessing its values. It is used to represent table or matrices.

4) Write a statement that declares an integer array of 10 element size and initializes it with values.

```
int array[10]={1,2,3,4,5,6,7,8,9,10};
```

5) How to initialize a one-dimensional array in C. Give an example.

It is a list of items that has one variable name and only one subscript to access the items.

Initialization of one dimensional array

- **datatype array_name[size]={list of values};**

Example:

```
int number[3] = {1,2,3};
```

This statement initializes integer variable number with 3 elements 1,2 and 3.

6) How to initialize a two-dimensional array in C. Give an example.

- Two-dimensional array

It is an array of arrays that has two subscripts for accessing its values. It is used to represent table or matrices.

Initialization of two dimensional array:

- **datatype array_name[row_size][column_size]={list of values};**

Example:

```
int table[2][3] = {0,0,0,1,1,1};
```

This statement initializes variable name **table** with row size 2 and column size 3 having 6 elements.

7) What is a string? How to declare a variable to hold string value in C?

- A string is a sequence of characters that is treated as a single data item.

Declaration of string Variable:

- **char string_name[size];**

Example:

```
char city[10];
```

8) What is the difference between reading string value using scanf("%s") and gets() functions?

scanf() function and gets() function are used to read the string.

scanf() function terminates its input on the first white space it ends. For example

```
char address[10];
```

```
scanf("%s",address);
```

If the text NEW YORK is read from the terminal then only the word NEW will be read into the array address.

Whereas gets() function does not terminate the string when it finds whitespace. It reads the characters from the keyboard until a new line character is encountered. For example

```
char address[10];
```

```
gets(address);
```

If the text NEW YORK is read from the terminal then the whole sentence will be read into the array address.

9) List any four-string handling functions available in C

- strcat()
- strcmp()
- strcpy()
- strlen()

10) List any four-character handling functions.

- isalpha()
- isdigit()
- isspace()
- islower()

11) How to declare and initialize string variable?

- The general form of declaration of a string variable is:

```
char string_name[ size];
```

The size determines the number of characters in the string_name. Some examples are as follows:

```
char city [10];
```

```
char name [30];
```

Here string name **city** has been declared as the character type with maximum size 10 and string name **name** as character type with max size 30.

Like numeric arrays, character arrays may be initialized when they are declared. C permits a character array to be initialized in either of the following two forms:

- char city [9] "NEW YORK";
- char city [9] 'N', 'E', 'W', ',', 'Y', 'O', 'R', 'K', '\0';

The reason that city had to be 9 elements long is that the string NEW YORK contains 8 characters and one element space is provided for the null terminator.

12) What are the possible values that the function strcmp() can return? What do they mean?

- **strcmp (string1,string2);**

This strcmp function compares two strings identified by the arguments. Major concern here is to determine whether the strings are equal.

(a) returns 0 if they are equal;

(b) If they are not equal then it returns the numeric difference between the first nonmatching characters in the string.

13) How to initialize a string array in C. Give an example.

C allows us to represent strings as character arrays. For Example

```
char city [9]={'N', 'E', 'W', ' ', 'Y', 'O', 'R', 'K', '\0'}
```

The reason that city had to be 9 elements long is that the string NEW YORK contains characters and one element space is provided for the null terminator. Note that when we initialize its elements, we must supply explicitly the null terminator in order to match the maximum elements.

14) Mention suitable string functions to do the following:

a. To find length of the string

➤ strlen

b. To copy one string to another string

➤ strcpy

c. To add two strings

➤ strcat

d. To reverse given string

➤ strrev

15) What is pointer? Specify its use.

A pointer is a variable that stores the address of another variable.

Uses:

- Pointers can be used to return multiple values from a function via function arguments.
- Pointers permit references to functions and thereby facilitating passing of functions as arguments to other functions.
- The use of pointer arrays to character strings results in saving of data storage space in memory.
- Pointers allow C to support dynamic memory management.
- Pointers provide an efficient tool for manipulating dynamic data structures such as structures, linked lists, queues, stacks, and tree.

16) List any four advantages of pointers

- Pointers make the programs simple and reduce their length.
 - Pointers are helpful in allocation and de-allocation of memory during the execution of the program.
- Thus, pointers are the instruments of dynamic memory management.
- Pointers enhance the execution speed of a program.
 - Pointers save the memory.

17) Specify how to declare a pointer variable and initialization of it?

Declaration of Pointer variables:

➤ **data_type*pt_name;**

Initialization of Pointer variable:

```
int quantity;
```

```
int* p;
```

```
p=&quantity;
```

18) List any four rules to be followed when performing operations on pointer variable

- A pointer variable can be assigned the address of another variable.
- A pointer variable can be assigned the values of another pointer variable.
- A pointer variable can be initialized with NULL or zero value.
- A pointer variable can be pre-fixed or post-fixed with increment or decrement operators.

19) What do you mean by scale factor in pointer increment?

When a pointer is incremented, its value is increased by the 'length' of the data type it points to. This length is called the scale factor. For example if p is an integer pointer with an initial value of 2800 then after the operation $p=p+1$ ($p++$), the value of P will be 2804.

6 MARKS

1) Explain one-dimensional array with the help of suitable code example.

A list of items can be given a single variable name using only one subscript and such a variable is called as a single subscripted variable or an one dimensional variable.

Number of subscripts determines the dimensions of the array.

Declaration of one dimensional array

General form :

datatype variable_name[size];

The datatype specifies the type of element that will be contained in the array, such as int, float or char.

variable _name is the name of the array.

Size indicates the maximum number of elements that can be stored inside the array.

Example :

int number[3];

declares the number to be an array of 3 integer elements. Any subscripts 0 to 2 are valid. Similarly

char name[10];

declares the name as the character array(string) variable that can hold a maximum of 10 characters.

Suppose we declare the array of 5 elements

int number[5];

The computer will reserve the five storage locations.

This would cause the array number to store the values as shown below:

35	number[0]
40	number[1]
20	number[2]
57	number[3]
19	number[4]

If we read the string “WELL DONE”

Each character of the string is treated as an element of the array and will be stored in the memory as follows:

‘W’	‘E’	‘L’	‘L’	‘ ’	‘D’	‘O’	‘N’	‘E’
-----	-----	-----	-----	-----	-----	-----	-----	-----

Initialization of one dimensional arrays

After an array is declared, its elements must be initialized.

The general form is :

datatype array_name[size] = {list of values};

The values in the list are separated by commas. For example

int number[3] = { 1,2,3}

will declare the variable number as an array of size 3 and assign values 1,2 and 3 respectively. If the number of values in the list is less than the number of elements then only those many elements will be initialized and rest of the elements will be set to zero automatically.

Character array can also be initialized in the same manner.

char name[4] = { ‘J’ , ‘O’ , ‘H’ , ‘N’ };

this declares the name to be an array of the characters, initialized with the string “JOHN”.

We can also assign this string directly

char name[4] = “JOHN”;

Example program: (Only if asked)

```
main()
{
int number[3]={1,2,3};
printf(“The first element is %d”,array[0]);
printf(“The second element is %d”,array[1]);
```

```
printf("The third element is %d",array[2]);
return 0;
}
```

Output:

The first element is 1
The second element is 2
The third element is 3

2) Explain two stages of initialization of in one dimensional array.

Initialization of one dimensional arrays

After an array is declared, its elements must be initialized.

An array can be initialized at two stages

- At compile time
- At run time

Compile Time Initialization

Compile time initialization is initializing the elements at the declaration part itself same as the ordinary variable declaration at the compile time.

The general form is :

datatype array_name[size] = {list of values};

The values in the list are separated by commas. For example

int number[4] = { 1,2,3,4}

will declare the variable number as an array of size 4 and assign values 1,2,3 and 4 respectively. If the number of values in the list is less than the number of elements then only those many elements will be initialized and rest of the elements will be set to zero automatically.

Character array can also be initialized in the same manner.

char name[4] = { 'J' , 'O' , 'H' , 'N' };

this declares the name to be an array of the characters, initialized with the string "JOHN".

We can also assign this string directly

char name[4] = "JOHN";

Example:

```
main()
{
int array[3]={1,2,3};
printf("The first element is %d",array[0]);
printf("The second element is %d",array[1]);
printf("The third element is %d",array[2]);
return 0;
}
```

Output:

The first element is 1
The second element is 2
The third element is 3

Run time initialization

Run time initialization is initializing the elements at the terminal(output screen) with the use of input statements such as scanf().

For example

```
int x[3];
scanf("%d%d%d", &x[0], &x[1], &x[2]);
```

will initialize array elements with the values entered through the keyboard in the output screen.

This explicit initialization approach is usually used for initializing the large arrays.

Example:

```
main()
{
int array[3],i,n;
printf("Enter the size of the array : ");
scanf("%d",&n);
printf("Enter the elements : ");
for(i=0;i<n;i++)
scanf("%d",&array[i]);
printf("The entered elements are %d",array[i]);
return 0;
}
```

Output

Enter the size of the array: 3

Enter the elements : 1 2 3

The entered elements are 1 2 3

3) Explain two-dimensional array with the help of suitable code example.

Two dimensional array is an array having two subscripts which can be used to represent matrices which is a collection of rows and columns.

Number of subscripts determines the dimensions of the array.

Declaration of two dimensional array

General form :

datatype array_name[row_size][column_size];

The datatype specifies the type of elements that will be contained in the array, such as int, float or char.

variable _name is the name of the array.

Row size and column size indicates the maximum number of elements that can be stored inside the rows and columns of the array respectively.

Example :

int number[2][2];

declares the number to be an array of 4 integer elements. Where the number of rows are 2 and columns are 2.

When we declare the two dimensional array of row size 2 and column size 2.

int number[2][2];

The computer will allocate memory as follows:

	Column0	Column1	Column2
	↓	↓	↓
	[0][0]	[0][1]	[0][2]
Row 0 →	12	13	14
	[1][0]	[1][1]	[1][2]
Row 1 →	17	15	34
	[2][0]	[2][2]	[2][2]
Row 2 →	34	34	23

Initialization of two dimensional arrays

Two dimensional arrays may be initialized by following their declaration with a list of initial values enclosed in braces.

The general form is :

datatype array_name[row_size][column_size] = {list of values};

datatype is the type of elements. array_name is the name of the array. row_size and column_size is the size of rows and columns respectively. List of values are the initialized values. The values in the list are separated by commas. For example

```
int number[2][2] = { 1,2,3,4};
```

or

```
int number[2][2] = {  
    {1,2},  
    {3,4},  
};
```

will declare the variable **number** of integer type as an array of row size 2 and column size 2 assigning values 1,2,3&4. If the number of values in the list is less than the number of elements then only those many elements will be initialized and rest of the elements will be set to zero automatically.

Example program: (Only if asked)

```
main()  
{  
int array[2][2]={1,2,3,4};  
printf("The first element is %d",array[0][0]);  
printf("The second element is %d",array[0][1]);  
printf("The third element is %d",array[1][0]);  
return 0;  
}
```

Output:

The first element is 1

The second element is 2

The third element is 3

4) Explain the different ways available for the initialization of string variable.

C allows us to represent strings as character arrays. String variable is always declared as an array of characters. The general form of declaration of string variable is :

```
char variable_name[size];
```

char is the datatype of the variable_name. size determines the number of characters a string name.

Examples are as follows :

```
char city[10];  
char name[30];
```

city is the name of the string of character type which can have maximum 10 characters.

name is the name of the string of character type which can have maximum 30 characters.

Like numeric arrays, character arrays may be initialized when they are declared.

When the compiler assigns a character string to a character array , it automatically supplies a null character ('\0') at the end of the string.

C permits the initialization of the string variable in either of the following two ways:

```
char city[9] = "NEW YORK";
```

```
char city[9] = {'N','E','W',' ','Y','O','R','K','\0'};
```

The string NEW YORK contains 8 characters with a space element. When we initialize a string by listing its elements we must pass an explicit null character to match the size of the array.

Whereas when we directly initialize the string as "NEW YORK" we need to pass null character, remaining elements will be automatically set to zero to match the size.

C also permits to initialize string without specifying the number of elements. In such cases size of the array will be determined automatically, based on the number of elements initialized.

Example :

```
char string[] = {'G','O','O','D','\0'} ;
```

defines an array string as a five element array.

We can also declare the size much larger than the string size in the initializer.

```
char string[10] = "GOOD";
```

is permitted. The computer will create a character array of size 10, places the value “GOOD” in it and initializes all other elements to NULL.

G	O	O	D	\0	\0	\0	\0	\0	\0
---	---	---	---	----	----	----	----	----	----

char str[3] = “GOOD”; This will be illegal.

5) Write a program to sort elements of an integer array.

```
#include<stdio.h>
main()
{
int array[100];
int i,j,n,temp;
clrscr();
printf("Enter the array size\n");
scanf("%d",&n);
printf("Enter the array elements one by one\n");
for(i=0;i<n;i++)
{
scanf("%d",&array[i]);
}
printf("Unsorted array is\n");
for(i=0;i<n;i++)
{
printf("%d\n",array[i]);
}
for(i=0;i<n;i++)
{
for(j=0;j<(n-i-1);j++)
{
if(array[j]>array[j+1])
{
temp=array[j];
array[j]=array[j+1];
array[j+1]=temp;
}
}
}
}
```

6) Write a program to search for an element in an integer array.

```
#include<stdio.h>
main()
{
int a[100],i,n,ele;
clrscr();
printf("Enter the size of the array:\n");
scanf("%d",&n);
printf("Enter the elements in an array:\n");
for(i=0;i<n;i++)
{
scanf("%d",&a[i]);
}
printf("Enter the element to be searched : ");
scanf("%d",&ele);
i=0;
while(i<n && ele!=a[i])
{
}
```

```

i++;
}
if(i<n)
printf("Element found at position = %d",i);
else
printf("Element not found");
getch();
return 0;
}

```

7) Explain with example how to declare, initialize and use strings in C.

Declaration of string

C does not support strings as a data type. However, it allows us to represent strings as character arrays. A string variable is any valid C variable name and is always declared as an array of characters. The general form of declaration of a string variable is:

```
char string_name[ size];
```

The size determines the number of characters in the string_name. Some examples are as follows:

```
char city [10];
char name [30];
```

When the compiler assigns a character string to a character array, it supplies null character (\0) at the end of the string. Therefore, the size should be equal to the maximum the string plus one.

Initialization of string

Like numeric arrays, character arrays may be Initialized when they are declared. C permits a character array to be initialized in either of the two following forms:

```
char city [9]= "NEW YORK";
char city [9]={ 'N', 'E', 'W', ' ', 'Y', 'O', 'R', 'K', '\0' }
```

The reason that city had to be 9 elements long is that the string NEW YORK contains characters and one element space is provided for the null terminator. Note that when we initialize its elements, we must supply explicitly the null terminator in order to match the maximum elements.

C also permits us to initialize a character array without specifying the number the size of the array will be determined automatically, based on the number example, the statement.

```
char string [] = { 'G', 'O', 'O', 'D', '\0' };
```

defines the array string as a five element array. We can also declare the size much larger than the string size in the initialize

```
char str [10] = "GOOD";
```

The familiar input function scanf can be used with %s formal specification to read in a string of characters. Example

```
char address [10];
scanf("%s", address);
```

```

main()
{
char greeting[6] = { 'H', 'E', 'L', 'L', 'O', '\0' };
printf("Greeting message : %s",greeting);
return 0;
}

```

Output :- Greeting message : Hello

```

main()
{
char greeting[6] = "Hello";
printf("Greeting message : %s",greeting);
return 0;
}

```

Output :- Greeting message : Hello

Both are equivalent.

8) List and explain any four-string handling functions available in C.

- strcat()
- strcmp()
- strcpy()
- strlen()

strcat()

strcat() function: The strcat function joins two strings together. It takes the following form:

strcat(string1,string2);

string1 and string2 are character arrays. When the function strcat is executed, string2 is appended to string1. It does so by removing the null character at the end of string1 and placing string2 from there. The string at string2 remains unchanged. For example consider the following 2 strings.

part1=

V	E	R	Y		\0				
---	---	---	---	--	----	--	--	--	--

part2=

G	O	O	D	\0		
---	---	---	---	----	--	--

Execution of the statement **strcat(part1,part2);**

will result in:

part1=

V	E	R	Y		G	O	O	D	\0
---	---	---	---	--	---	---	---	---	----

part2=

G	O	O	D	\0		
---	---	---	---	----	--	--

strcmp()

The strcmp function compares two strings identified by the arguments and has a value 0 if they are equal, If they are not, it has the numeric difference between the first nonmatching characters in the strings. It takes the form:

strcmp(string1, string2);

string1 and string2 may be string variables or string constants. Examples are:

strcmp(name, name2);

strcmp(name, "John");

strcmp("Rom", "Ram");

Our major concern is to determine whether the strings are equal; if not, which is alphabetically above. The value of the mismatch is rarely important. For example, the statement

strcmp("their", "there");

will return a value of -9 which is the numeric difference between ASCII "i" and ASCII "r". That is, "i" minus , "r" in ASCII code is -9. If the value is negative, string1 is alphabetically above string2.

strcpy()

The strcpy function works almost like a string-assignment operator. It takes the following form:

strcpy(string1,string2)

and assigns the contents of string2 to string1. string2 may be a character array variable or a string constants.

For example, the statement

strcpy(string1, string2);

strcpy(city, "DELHI");

will assign the string "DELHI" to the string variable city. Similarly, the statement

strcpy(city1, city2);

will assign the contents of the string variable city2 to the string variable city1. The size of the array city1 should be large enough to receive the contents of city2.

strlen()

This function counts and returns the number of characters in a string. It takes the form

n = strlen(string);

Where n is an integer variable, which receives the value of the length of the string. The argument may be a string constant. The counting ends at the first null character.

For example:

char str[5]="GOOD";

int n;

n=strlen(str);

This will give the total number of characters of the string stored in **str**. That is the length of the string "GOOD". The output will be: **4**

9) List and explain any four-character handling functions available in C.

isalpha() :

isalpha(ch);

This function checks whether the character variable/constant contains alphabet or not.

isdigit() :

isdigit(ch);

This function checks whether the character variable/ constant contains digit or not.

isspace() :

isspace(ch);

This function checks whether the character variable/constant contains a space or not.

isascii() :

isascii(ch);

The isascii() function tests for an ASCII character. Nonzero if c is an ASCII character; otherwise, zero.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
char n;
```

```
printf("\nEnter a character");
```

```
n=getchar();
```

```
if(isalpha(n))
```

```
printf("Entered character is an alphabet");
```

```
if(isdigit(n))
```

```
printf("Entered character is a digit");
```

```
if(isalnum(n))
```

```
printf("Entered character is an alphabet or digit");
```

```
if(isspace(n))
```

```
printf("Entered character is a blank space");
```

```
if(isascii(n))
```

```
printf("Entered character is an ascii character ");
```

```
return 0;
```

```
}
```

isupper() :

isupper(ch);

This function checks whether the character variable/constant contains an capital letter alphabet or not.

islower() :

islower(ch);

This function checks whether the character variable/constant contains a lowercase alphabet or not.

toupper() :

toupper(ch);

This function converts lowercase alphabet into uppercase alphabet.

tolower() :

tolower(ch);

This function converts an uppercase alphabet into lowercase alphabet.

#include<stdio.h>

int main()

{

char n;

printf("Enter an alphabet");

n=getchar();

if(islower(n))

n=toupper(ch);

else

ch=tolower(ch);

printf("Now alphabet=%c",n);

return 0;

}

Output :

Enter a character : A

Now alphabet = a

10) What is a pointer variable? How is it declared, initialized and used in a program?

Pointer is a variable that stores the address of another variable.

Declaration of pointer variables

Pointer variables contain addresses that belong to separate data type, they must be declared as pointers before we use them.

General form :**data_type *pt_name;**

This tells the compiler three things about the variable pt_name.

1. The asterisk (*) tells that the variable pt_name is a pointer variable.
2. Pt_name needs a memory location.
3. pt_name points to a variable of type data_type.

Example :**int *p;**

declares the variable p as a pointer variable that points to an variable of integer data type.int refers to the data type of the variable being pointed to by p and not the type of the value of the pointer.

Similary

float *x;

declares x as a pointer to a floating type variable.

Initialization of pointer variables

The process of assigning the address of a variable to a pointer variable is known as initialization.

Once a pointer variable has been declared we can use the assignment operator to initialize the variable.

Example :**int quantity;** //declaration of normal variable

```
int *p;           //declaration of pointer variable
p = &quantity;    //initialization
```

We can also combine the initialization with the declaration. That is,

```
int *p = &quantity;
```

is allowed. The only requirement is here is that the variable quantity must be declared before the initialization takes place. This is an initialization p not *p;

Accessing a variable through its pointer

```
int quantity,*p,n;
quantity = 120;
p = &quantity;
n=*p;
```

The first line declares quantity and n as integer variables and p as a pointer variable pointing to an integer. The second line assigns the value 120 to quantity and the third line assigns the address of quantity to the pointer variable p. The fourth line contains the indirection operator. When the operator * is placed before a pointer variable in an expression, the pointer returns the value of the variable of which the pointer value is the address. In this case, *p returns the value of the variable quantity, because p is the address of quantity. The * can be remembered as 'value at address'. Thus, the value of n would be 120.

n=*p; is equivalent to n=quantity;

Example program : (Only if asked in the question)

```
main()
{
int c,*pc;
c=2;
pc=&c;
printf("Address of c is : %u",&c);           //2000
printf("Value of c is %d",c);               //2
printf("Address of pointer pc is : %u",pc);  //2000
printf("Value of pointer pc is : %d",*pc);   //2
return 0;
}
```

11) Discuss the rules applied when performing operations on pointer variable.

1. A pointer variable can be assigned the address of another variable.
2. A pointer variable can be assigned the values of another pointer variable.
3. A pointer variable can be initialized with NULL or zero value.
4. A pointer variable can be pre-fixed or post-fixed with increment or decrement operators.
5. An integer value may be added or subtracted from a pointer variable.
6. When two pointers point to the same array, one pointer variable can be subtracted from another.
7. When two pointers point to the objects of the same data types, they can be compared using relational operators.
8. A pointer variable cannot be multiplied by a constant.
9. Two pointer variables cannot be added.
10. A value cannot be assigned to an arbitrary address (i.e., &x= 10; is illegal).

12) Illustrate the use of pointers with arrays.

When an array is declared, the compiler allocates a base address and sufficient amount of storage to contain all the elements of the array in memory locations. The base address is the location of the first element (index 0) of the array.

Suppose we declare an array x as follows:

```
int x[5] = { 1,2,3,4,5};
```

Suppose the base address is 1000 and assuming that each integer requires 4 bytes, the five elements will be stored as follows:

Positions→	x[0]	x[1]	x[2]	x[3]	x[4]
Values →	1	2	3	4	5
Address→	1000	1004	1008	1012	1016

Let us declare a pointer variable p that points to array variable x.

```
int *p , x[5];  
p = &x[0];
```

Here the pointer variable stores the address of the first element(base address) in an array x[0], the value becomes 1000.

Now we can access every value of x using p++ to move from one element to another.

```
p = x[0] -> 1000  
p+1 = x[1] -> 1004  
p+2 = x[2] -> 1008  
p+3 = x[3] -> 1012  
p+4 = x[4] -> 1016
```

The address of the elements is calculated using its index and the scale factor of the data type. For example,

$$\begin{aligned}\text{address of } x[3] &= \text{base address} + (3 * \text{scale factor of int}) \\ &= 1000 + (3 * 4) \\ &= 1012\end{aligned}$$

To access array elements we can use pointers instead of array index, i.e. *(p+3) gives the value of x[3].

(Only if asked) Example :

```
main()  
{  
int x[5] = { 1,2,3,4,5};  
int *p;  
p = x[0];  
printf("Value of p is %d", *p);  
printf("Value of (ptr+1) is %d", *(p+1));  
printf("Address of p is %u", p);  
printf("Address of (ptr+1) is %u", (p+1))  
return 0;  
}
```

13) What do you mean by pointer arithmetic? Explain how it is differed from other integer arithmetic.

A pointer in c is an address, which is a numeric value. Therefore, you can perform arithmetic operations on a pointer just as you can on a numeric value. There are four arithmetic operators that can be used on pointers: ++, --, +, and -.

We can't perform every type of arithmetic operations with pointers. Pointer arithmetic is slightly different from arithmetic we normally use in our daily life. The only valid arithmetic operations applicable on pointers are:

1. Addition of integer to a pointer
2. Subtraction of integer to a pointer
3. Subtracting two pointers of the same type

For example, if we have an integer variable p which contains address 1000, then on incrementing p++ will be 1001.

But the pointer arithmetic is performed relative to the base type of the pointer.

For example if we have to increment a pointer variable p whose address is 1000 then p++ will be 1004 (i.e. $1000 + 1 * 4$) because the size of the int data type is 4 bytes.

Arithmetic operation on type char seems like ordinary arithmetic because the size of char type is 1 byte. Another important point to note is that when we increment and decrement pointer variable by adding or subtracting numbers then it is not necessary that the pointer variable still points to a valid memory location.

Like other variables ,pointer variables can be used in expressions. For example if p1 and p2 are properly declared and initialized pointers, then the following statements are valid:

```
y=*p1 * *p2;
```

```
sum = sum + *p1;
z=5* - *p2/ *p1;
*p2 = *p2 + 10;
```

Note that there is a blank space between / and * in the item 3 above. The following is wrong:

```
z=5* - *p2 / *p1;
```

The symbol /* is considered as the beginning of a comment and therefore the statement fails.

C allows us to add integers to or subtract integer from pointers, as well as to subtract one pointer from another.

$p1 + 4$, $p2 - 2$, and $p1 - p2$ are all allowed. If $p1$ and $p2$ are both pointers to the same array, then $p2 - p1$ gives the number of elements between $p1$ and $p2$.

In addition to arithmetic operations discussed above, pointers can also be compared the relational operators. The expression such as $p1 > p2$, $p1 == p2$, and $p1 != p2$ are allowed. However, any comparison of pointers that refer to separate and unrelated variables makes no sense. Comparisons can be used meaningfully in handling arrays and strings.

We may not use pointers in division or multiplication.

For example, expression such as

$p1 / p2$ or $p1 * p2$ or $p1 / 3$

are not allowed. Similarly, two pointers cannot be added. That is, $p1 + p2$ is illegal.

14) Explain how pointers are used to access string.

Strings are treated like character arrays and therefore they are declared and initialized as follows:

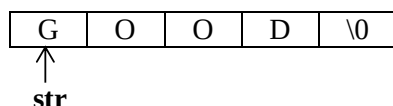
```
char str[5]="good";
```

The compiler automatically inserts the null character '\0' at the end of the string.

C supports an alternate method to create strings using pointer variables of type **char**.

Example : `char *str = "good";`

This creates a string and stores its address in the pointer variable **str**. The pointer **str** now points to the first character of the string "good" as:



Suppose we declare an char array x as follows:

```
char x[5] = { 'G','O','O','D','\0' };
```

Suppose the base address is 1000 and assuming that each character requires 1 byte, the 4 elements will be stored as follows:

Positions→	x[0]	x[1]	x[2]	x[3]
Values →	G	O	O	D
Address→	1000	1001	1002	1003

We can also give values to a string pointer in a new line.

Example : `char *str;`
`str = "good";`

We can print the content of the string **str** using either `printf` or `puts` functions as follows:

```
printf("%s",str);
puts(str);
```

In C , a constant character string always represents a pointer to that string. And therefore the following statements are valid:

```
char *name;
name = "Dehi";
```

These statements will declare **name** as a pointer of character type and assign the constant string to **name**.

This

type of assignment will not work with character arrays. For example

```
char name[20];
name = "Delhi"; will not work.
```

15) Write a note on pointers increment scale factors.

We have seen that the pointers can be incremented like

```
p1=p2+2;
```

```
p1=p1+1;
```

And so on. However an expression like

```
p1++;
```

will cause the pointer p1 to point to the next value of its type. for example, If p1 is an integer, pointer with an initial value 2800 then after the operation p1=p1+1, the value of p1 will be 2804 not 2801. That is when we increment a pointer its value is increased by “length” of the data type. This length is called the scale factor.

The lengths of various data types are as follows:

Char	1byte
int	4bytes
float	4bytes
long int	8bytes
double	8 bytes

The number of bytes used to store various data types depends on the system and can be found by making use of the sizeof operator. For example, if x is a variable, the size(x) returns the number of bytes needed for the variable.

A pointer variable can be assigned address and value of another variable. It can be pre-fixed or post-fixed with increment or decrement operators. An integer value may be added or subtracted from a pointer variable. When two pointers point to the same array, one pointer variable can be subtracted from another. When two pointers point to the object of the same data type, they can be compared using relational operators. A pointer variable cannot be multiplied by a constant. Two pointer variables cannot be added. a value cannot be assigned to an arbitrary address.